



LIONSDEV

Curso de Programação

MÓDULO 4

Domínio do Git e GitHub

Professor Nicolas Cardoso Motta



Revisando o Módulo 3



Requisitos e escopo

Antes do código: o que está dentro e o que está fora do projeto.



Menu em laço

`while` + `switch`
mantêm o programa rodando até o usuário escolher sair.



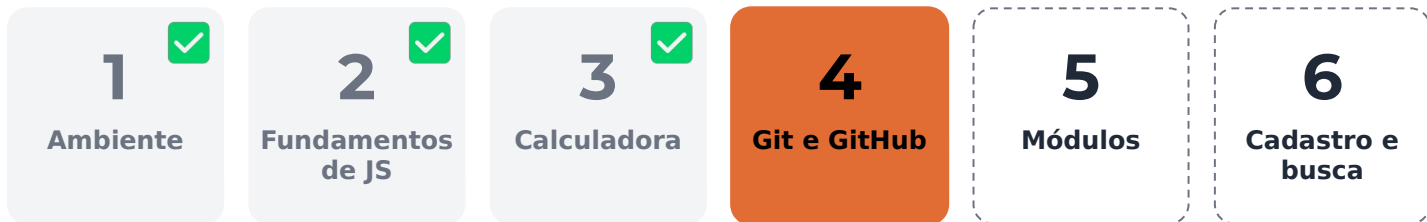
Validação e refatoração

`isNaN` barra a entrada inválida; funções como `lerNumero` evitam repetir código.

Pergunta para abrir: o que acontece na calculadora se o usuário digitar `abc` no lugar de um número?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND



FRONTEND E PROJETO

Você está aqui: **Módulo 4**, Git e GitHub



Objetivo

- Entender o que é controle de versão e a diferença entre Git e GitHub.
- Instalar e configurar o Git no Ubuntu.
- Criar repositórios locais e remotos e sincronizá-los com push, pull e clone.
- Trabalhar com branches, pull requests e merge.
- Organizar os projetos com .gitignore e README.md.

Até agora, cada atividade vivia só no seu computador. E se o PC der problema? E se duas pessoas precisarem mexer **no mesmo código?**



Git

Controle de versão

O que é o Git?



Rastreia alterações

Um sistema de controle de versão: registra cada mudança no código durante o desenvolvimento.



Colaboração

Várias pessoas trabalham no mesmo projeto, ao mesmo tempo, de forma organizada.



Histórico completo

Compara versões, volta para versões anteriores e junta o trabalho de cada pessoa com segurança.



Git x GitHub



Git

**Ferramenta de linha de comando,
instalada no seu computador.**

**Versiona o projeto localmente,
mesmo sem internet.**



GitHub

**Plataforma na nuvem, construída
em torno do Git.**

**Hospeda os repositórios e tem
issues, pull requests, wikis e
gestão de projetos.**

**O Git é a ferramenta; o GitHub é um dos lugares onde guardamos os
projetos versionados.**

Onde hospedar os repositórios?



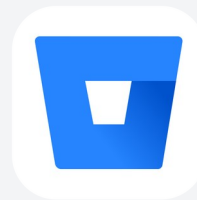
GitHub

O mais popular. É o que vamos usar.



GitLab

Muito usado em empresas, com CI/CD integrado.



Bitbucket

Da Atlassian, integrado ao Jira.

Instalando o Git

```
$ sudo apt update  
$ sudo apt install git  
$ git --version  
git version 2.x.x
```

apt update → atualiza a lista de pacotes do Ubuntu.

apt install git → instala o Git.

git --version → confere se deu certo.

Atividade Prática 1

GitHub

1. Acesse `github.com` e crie a sua conta.
2. Use o seu e-mail, crie uma senha e um nome de usuário único.
3. Siga os colegas que quiser.



Dica: guarde o e-mail que você usou: ele vai ser usado na configuração do Git.

Configurando o Git

```
$ git config --global user.name "Seu Nome"  
$ git config --global user.email "seu@email.com"  
$ git config --list
```

user.name / email → aparecem em cada commit que você fizer.

mesmo e-mail → use o e-mail da sua conta do GitHub: é assim que os commits aparecem no seu perfil.

--global → vale para todos os repositórios do computador.

--list → mostra as configurações para você conferir.

Na rede da Lions: o proxy

Os computadores da Lions acessam a internet por um proxy. Sem configurar, o Git não conecta ao GitHub:

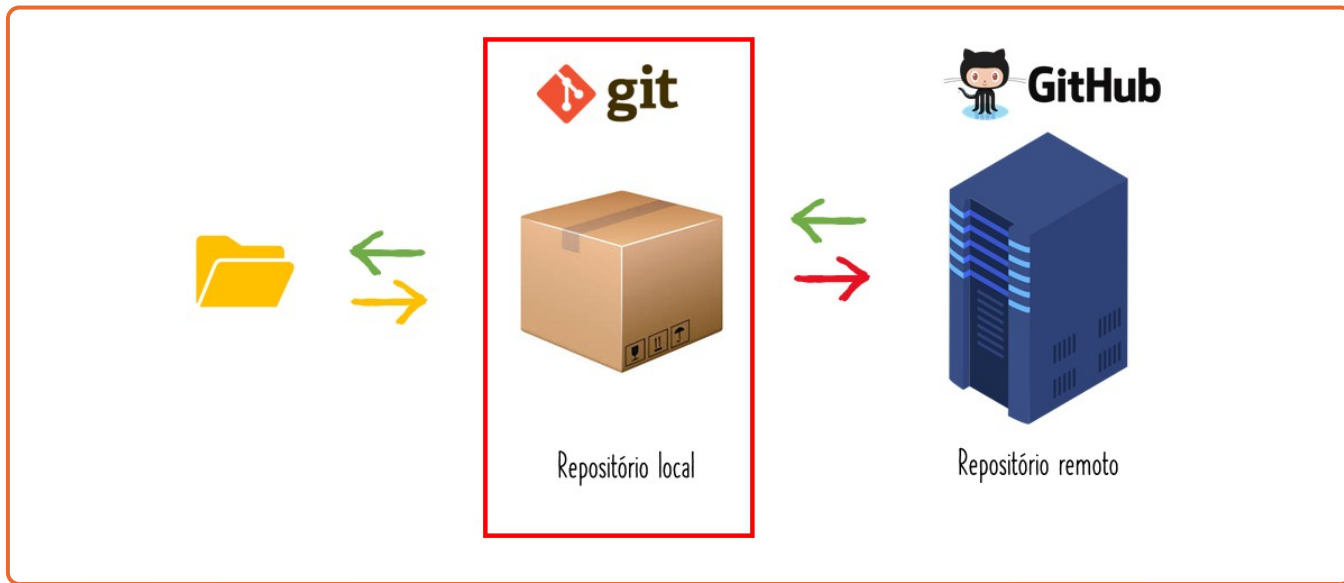
```
$ git config --global http.proxy http://nome:senha@192.168.10.1:3128  
$ git config --global https.proxy http://nome:senha@192.168.10.1:3128
```

O nome e a senha são os do computador da Lions, não os do GitHub.



Repositório Local

Do seu computador para o GitHub



Arquivos na pasta → repositório local (Git) → repositório remoto (GitHub)

As três áreas do Git

Do arquivo editado até o histórico, as mudanças passam por três lugares:



Diretório

Working Directory: onde você edita os arquivos.



Palco

Staging Area: o que vai entrar no próximo commit.



Repositório

Git Repository: o histórico de commits.

git add leva do diretório para o palco. **git commit** leva do palco para o repositório.

Criando o primeiro repositório

```
$ mkdir lions-dev  
$ cd lions-dev  
$ git init  
$ git status  
$ git add .  
$ git commit -m "Primeiro commit"
```

git init → transforma a pasta num repositório (cria a pasta oculta **.git**).

git status → mostra o que mudou e o que está no palco.

git add . → coloca todas as mudanças no palco.

git commit -m → registra o que está no palco no histórico, com uma mensagem.



Cuidado: git init uma vez só, na pasta certa

Rodar **git init** na pasta pessoal, ou dentro de outro repositório, faz o Git vigiar arquivos que não deveria.

- Confira com **pwd** antes: um projeto, um git init.
- Um repositório dentro de outro vira bagunça no **git status**.
- Errou? Apague a pasta oculta **.git** daquela pasta com **rm -rf .git**.

```
$ pwd  
/home/aluno/Documentos/lions-dev  
$ git init
```

Lendo o git status

```
$ git status
```

```
On branch main
```

```
Changes to be committed:
```

```
    new file:   README.md
```

```
Changes not staged for commit:
```

```
    modified:   calculadora.js
```

```
Untracked files:
```

```
    senhas.txt
```

to be committed → já está no palco: entra no próximo commit.

not staged → foi modificado, mas ainda não passou pelo **git add**.

untracked → arquivo novo que o Git ainda não acompanha.

Boas mensagens de commit

Evite

Mensagens que não dizem nada sobre a mudança.

```
git commit -m "update"  
git commit -m "arrumei"  
git commit -m "asdf"
```

Prefira

Mensagens que explicam o que o commit faz.

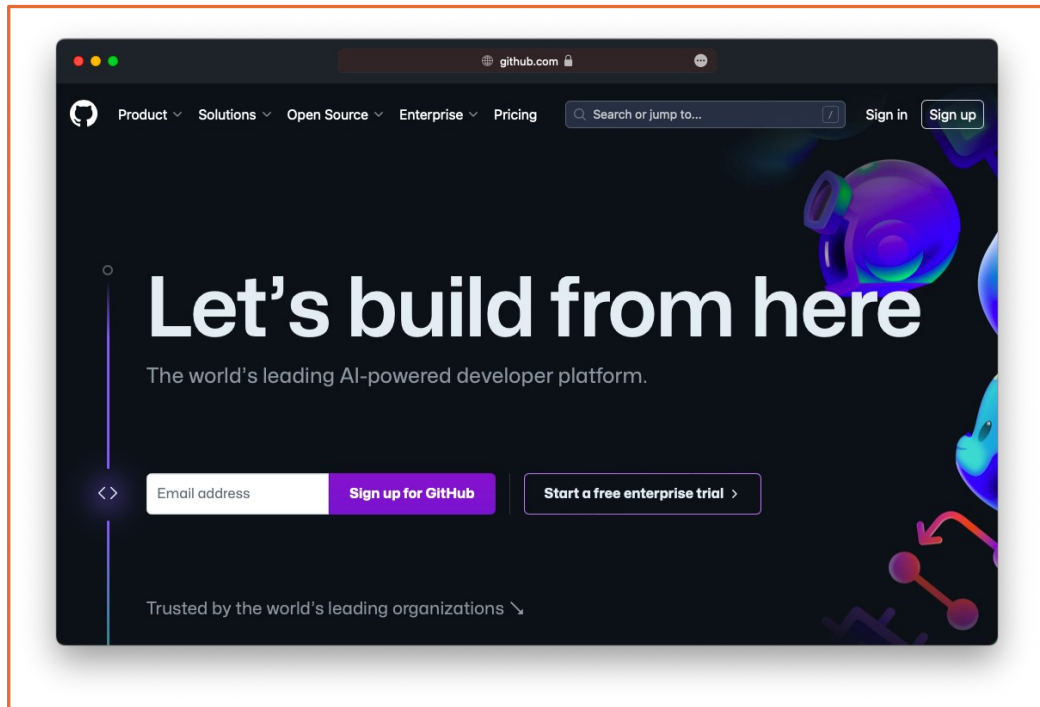
```
git commit -m "cria menu da calculadora"  
git commit -m "trata divisão por zero"  
git commit -m "adiciona README"
```

Daqui a um mês, o histórico precisa contar a história do projeto para você e para a equipe.



Repositório Remoto

GitHub



- **Guarda uma cópia do repositório na nuvem.**
- **Funciona como backup e como vitrine: recrutadores olham o seu GitHub.**
- **É onde a equipe colabora, com pull requests e revisões.**

Atividade Prática 2

GitHub

1. No GitHub, clique em New repository.
2. Dê um nome, deixe público e crie o repositório vazio.
3. Copie a URL HTTPS que aparece na página.



Cuidado: não marque "Add a README" nem ".gitignore" na criação: com o remoto vazio, o primeiro push não dá conflito.

Conectando e enviando

```
$ git remote add origin https://github.com/usuario/lions-dev.git  
$ git branch -M main  
$ git push -u origin main
```

remote add origin liga o repositório local ao do GitHub. **branch -M main** garante que a branch se chama main. O **-u** do push memoriza o destino: nas próximas vezes, basta **git push**.



Cuidado: master ou main?

Dependendo da configuração, o `git init` cria a branch **master**, e o `git push -u origin main` falha.

- O erro é `src refsPEC main does not match any`: não existe branch **main** no seu computador.
- Renomeie antes do push com `git branch -M main`.
- Ou configure de uma vez: `git config --global init.defaultBranch main`.

```
$ git push -u origin main
error: src refsPEC main does not match any
```



Cuidado: a senha do GitHub não funciona no push

O GitHub não aceita a senha da conta no terminal.

- Quando o terminal pedir a senha, use um **Personal Access Token**: no GitHub, em **Settings > Developer settings > Personal access tokens**.
- Guarde o token como uma senha: ele dá acesso à sua conta.

```
Username for 'https://github.com': aluno  
Password for 'https://aluno@github.com':      # cole o token aqui
```

Por que fazer push?



Envio

Os commits locais vão para o repositório remoto.



Atualização

O remoto fica igual às suas branches locais.



Backup

O código fica seguro mesmo se algo acontecer com o computador.



Colaboração

A equipe vê e usa o seu trabalho; conflitos aparecem cedo.

Atividade Prática 3

Repositório do curso

1. Navegue até Documentos e crie a pasta lions-dev.
2. Dentro dela, crie uma pasta para cada módulo e coloque as atividades.
3. Na pasta lions-dev, rode `git init`, `git add .` e `git commit -m "..."`.
4. Crie o repositório remoto vazio no GitHub e conecte com `git remote add origin`.
5. Envie com `git push -u origin main`.



Pull e Clone

Push x Pull



git push: "empurrar"

Envia as mudanças locais para o repositório remoto.

Do computador para o GitHub.



git pull: "puxar"

Traz as mudanças remotas para o repositório local.

Do GitHub para o computador.

Exemplo: fiz um programa no computador do trabalho e dei push. Em casa, dei pull e continuei de onde parei.



Cuidado: push rejeitado

Se o GitHub tem commits que você não tem, o push é **rejeitado.**

- Acontece quando outra pessoa, ou você mesmo pelo site, mudou o repositório remoto.
- Solução: **git pull** primeiro, depois **git push**.

```
$ git push
! [rejected]          main -> main (fetch first)
error: failed to push some refs
```

Atividade Prática 4

Pull

1. Pelo site do GitHub, altere uma das atividades do seu repositório.
2. No terminal, rode `git pull` para trazer a versão nova para o seu computador.
3. Abra o arquivo no VS Code e confira a mudança.

Clone: baixando um repositório

```
$ git clone https://github.com/usuario/repositorio.git  
$ cd repositorio  
$ npm install
```

Use a URL https://: a versão SSH (git@github.com:...) não funciona na rede da Lions. E a node_modules não vem junto: depois de clonar, rode npm install.

Atividade Prática 5

Clone

1. Clone algum repositório público do GitHub, ou o repositório do curso: `github.com/nicolassmotta/curso-lions-dev`.
2. Entre na pasta clonada e liste os arquivos com `ls`.

Fork



Uma cópia na sua conta

O fork copia um repositório para a sua conta do GitHub. O clone copia para o seu computador.



Mexa sem medo

Você altera o projeto sem afetar o repositório original.



Código aberto

É assim que se contribui com projetos de outras pessoas: fork, alteração e pull request.

Atividade Prática 6

Fork

1. Faça um fork de um repositório público. Sugestão: `github.com/iuricode/padroles-de-commits`.
2. Confira que ele apareceu nos repositórios da sua conta.



Dica: aproveite para ler o conteúdo: ele ensina um padrão de mensagens de commit usado no mercado.



Branches

O que é uma branch?



Linha independente

Uma linha de desenvolvimento separada, no mesmo projeto.



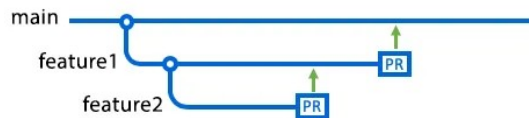
Sem mexer no principal

Novas funcionalidades e correções não interferem no código da main.



Trabalho em equipe

Cada pessoa trabalha na sua branch e, no fim, as mudanças são mescladas.



Comandos de branch

```
$ git branch # lista as branches
$ git checkout -b raiz-quadrada # cria e já muda para ela
$ git checkout main # volta para a main
$ git branch -d raiz-quadrada # apaga (se já foi mesclada)
$ git branch -D raiz-quadrada # força a exclusão
```

Nas versões novas do Git também existe git switch: git switch -c nome cria e muda, git switch main volta. Faz a mesma coisa.

Atividade Prática 7

Branch

1. No projeto da calculadora, crie a branch raiz-quadrada com `git checkout -b raiz-quadrada`.
2. Crie uma função que calcula a raiz quadrada do resultado atual, o case no switch e a opção no menu.
3. Use `git add .`, `git commit -m "..."` e `git push -u origin raiz-quadrada`.

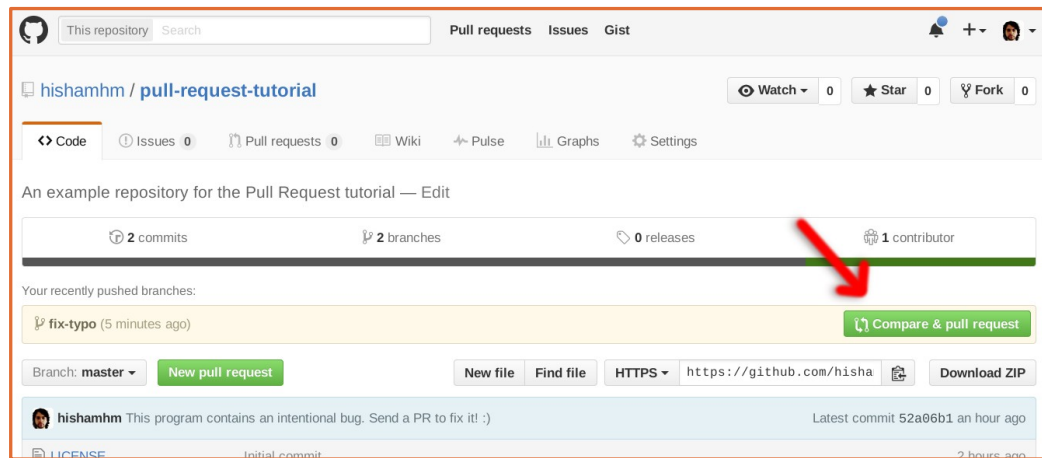


Dica: o push precisa do nome da branch, mesmo você estando nela: na primeira vez, o GitHub ainda não conhece a raiz-quadrada.



Pull Request e Merge

Pull Request



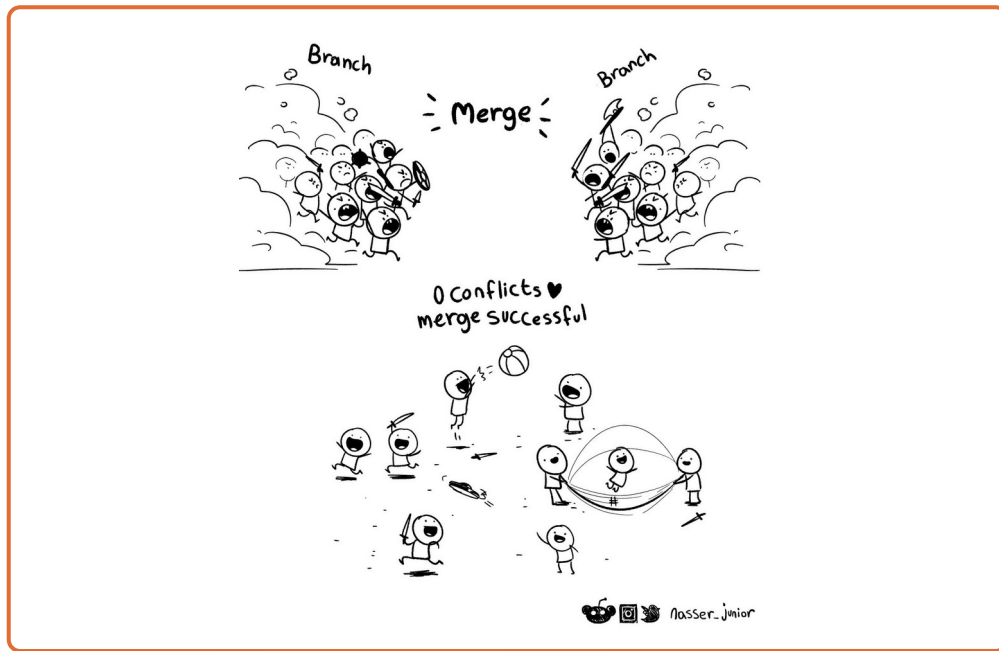
- Um pedido para mesclar as mudanças de uma branch em outra, geralmente na **main**.
- A equipe revisa o código antes de ele entrar no projeto.
- É o lugar para discutir, comentar e pedir ajustes.

Atividade Prática 8

Pull Request

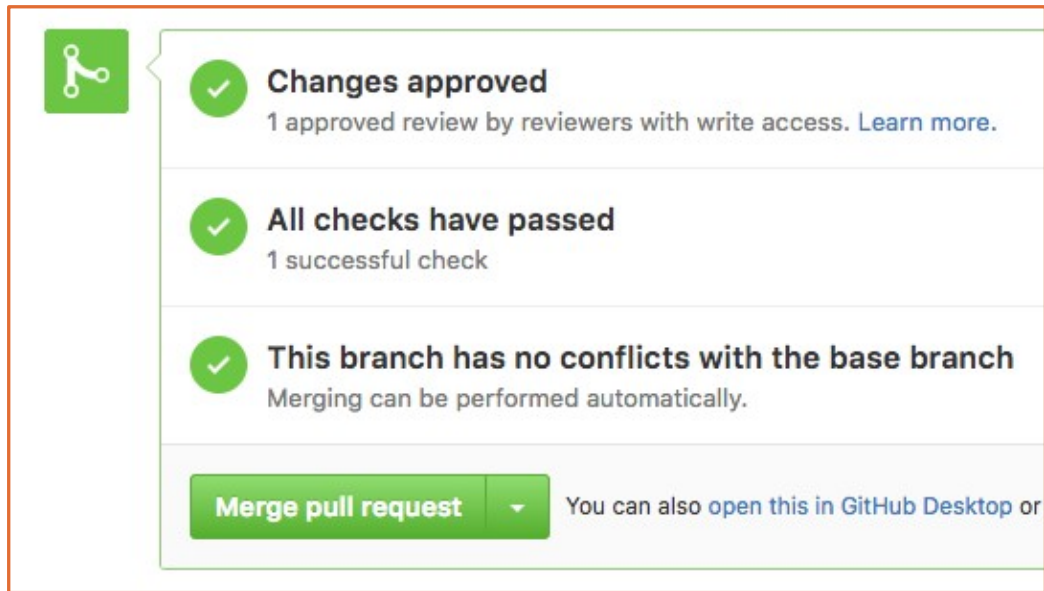
- 1. No GitHub, abra o repositório da calculadora e clique em Compare & pull request.**
- 2. Descreva as mudanças que você fez.**
- 3. Solicite a revisão de um colega.**

Merge: juntando as branches



O que essa charge diz sobre o merge?

Como funciona o merge no GitHub



- O pull request aprovado pode ser mesclado na branch de destino.
- Se houver conflito com a main, o GitHub avisa e só libera o merge depois de resolvido.
- Quando está tudo certo, o botão Merge pull request fica verde.

Conflito: como ele aparece no arquivo

```
<<<<<< HEAD
console.log('Bem-vindo à calculadora!')
=====
console.log('Olá! Escolha uma operação.')
>>>>>> raiz-quadrada
```

<<<<<< HEAD → daqui até o
=====: a versão que já está na
branch atual.

>>>>>> → do **=====** até aqui: a
versão que está chegando.

resolver → escolha ou combine as
duas, apague as marcações e faça
git add e **git commit**.

git log: o histórico

```
$ git log --oneline
```

```
a1b2c3d adiciona raiz quadrada  
9f8e7d6 trata divisão por zero  
4c5d6e7 cria menu da calculadora  
1a2b3c4 Primeiro commit
```

git log → mostra todos os commits, do mais novo para o mais antigo.

--oneline → um commit por linha: o código (hash) e a mensagem.

mensagens → é aqui que as boas mensagens de commit fazem diferença.

Atividade Prática 9

Merge

1. Abra o pull request que você criou no GitHub.
2. Confira se o GitHub não aponta conflitos: se estiver tudo certo, o botão de merge fica liberado.
3. Clique em Merge pull request e confirme.
4. No terminal, volte para a principal com `git checkout main` e rode `git pull`.
5. Confira com `git log --oneline` que o commit da raiz quadrada está lá.



Desfazendo coisas

Todo mundo erra: o Git tem volta

Os comandos de desfazer

```
$ git diff # o que mudou (ainda fora do palco)
$ git restore arquivo.js # descarta a mudança no arquivo
$ git restore --staged arquivo.js # tira do palco, mantém a mudança
$ git commit --amend -m "nova msg" # corrige o último commit
$ git revert a1b2c3d # novo commit que desfaz outro
```

O `git restore arquivo.js` **apaga** as mudanças que não foram commitadas: não tem como recuperar depois.



Cuidado: não reescreva o que já foi para o GitHub

O **--amend** troca o último commit por outro. Se ele já foi enviado, o próximo push é **rejeitado**.

- Use o **--amend** só em commits que ainda estão no seu computador.
- Para desfazer algo que já está no GitHub, use o **git revert**: ele cria um commit novo e não mexe no histórico da equipe.



.gitignore e README.md

.gitignore

```
# .gitignore
node_modules/
.env
senhas.txt
```

.gitignore → um arquivo de texto que lista o que o Git deve ignorar e não mandar para o GitHub.

node_modules/ → nunca vai para o GitHub: é pesada e quem clonar recria com **npm install**.

senhas e chaves → arquivos como o **.env** (Módulo 9) ficam só no seu computador.

Atividade Prática 10

.gitignore

1. Crie um arquivo `.gitignore` no seu projeto.
2. Crie também um arquivo "de mentirinha", como `senhas.txt`.
3. Coloque o nome `senhas.txt` dentro do `.gitignore`.
4. Rode `git status` e repare que o Git finge que o arquivo nem existe.



Cuidado: o `.gitignore` só vale para arquivos que ainda não foram commitados. Se já foi para o GitHub, precisa remover do repositório.

README.md

```
# Calculadora LionsDev
```

```
Calculadora de terminal feita com Node.js.
```

```
## Como rodar
```

```
npm install  
node calculadora.js
```

```
## Funcionalidades
```

- Soma, subtração, multiplicação e divisão
- Porcentagem e resultado acumulado

README.md → o documento principal do projeto: é a primeira coisa que alguém lê no GitHub.

conteúdo → o que o programa faz, as ferramentas usadas e como rodar.

e - → é Markdown: **#** vira título e **-** vira lista.

Atividade Prática 11

README

- 1. Volte ao projeto da calculadora.**
- 2. Crie um arquivo README.md na pasta principal.**
- 3. Escreva uma documentação explicando o que a calculadora faz e como rodar.**
- 4. Faça o commit e o push.**

Cheat sheet do Git

Começar

```
git init  
git clone url
```

Ver o estado

```
git status  
git log --oneline
```

Salvar

```
git add .  
git commit -m "..."
```

Remoto

```
git remote add  
origin url
```

Enviar

```
git push  
git push -u origin x
```

Trazer

```
git pull
```

Branches

```
git branch  
git checkout -b x
```

Mudar de branch

```
git checkout main
```

Apagar branch

```
git branch -d x
```

Configurar

```
git config --global  
user.name "..."
```

Ignorar

```
.gitignore
```

Juntar

```
Pull request  
+ Merge no GitHub
```

Material do módulo



Aponte a câmera do celular

[github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo04](https://github.com/nicolassmotta/curso-lions-dev/tree/main/modulo04)



conteudo/
teoria comentada e o cheat sheet do módulo



exercicios_slides/
os exercícios feitos em aula



lista_de_exercicios/
listas para praticar em casa



Fim!

Daqui pra frente, toda atividade vai para o GitHub.